Implementación de pipeline de observabilidad end-to-end con OpenTelemetry y evaluación empírica de overhead en arquitecturas multi-cloud

Informe técnico de instrumentación SDK, Collector, correlación cross-signal y benchmark de rendimiento

Carlos Alberto Zambrano Braydi

0000395349

Henry Julián Salazar Salcedo

0000396117

Wolfran Alirio Pinzón Murillo

0000393439

Maestría en Arquitectura de Software, Universidad de La Sabana

Observabilidad en ambientes productivos — Unidad 2 — Grupo 11

Mg. María Fernanda Ochoa Paipilla

Octubre de 2026

Implementación de pipeline de observabilidad end-to-end con OpenTelemetry

Introducción

En los sistemas distribuidos basados en microservicios, el diagnóstico de incidentes se degrada con rapidez cuando las señales de telemetría residen en silos desconectados. Un fallo originado en una consulta lenta de base de datos suele propagarse aguas arriba manifestándose como un error 504 Gateway Timeout o una degradación en la experiencia del usuario, generando largas sesiones de depuración cuando el equipo carece de un hilo conductor que relacione los registros de log con el recorrido exacto de la petición (Sachto y Walcer, 2022).

El presente informe documenta la solución al Laboratorio 2 de la asignatura Observabilidad en Ambientes Productivos. El encargo consistió en diseñar e implementar un pipeline de observabilidad integral basado en el estándar abierto OpenTelemetry (OTel), que capture trazas distribuidas, métricas de rendimiento y logs estructurados desde dos microservicios dependientes (service-a y service-b) con persistencia en PostgreSQL, con despliegue objetivo en Google Cloud Platform (GKE) y Amazon Web Services (ECS Fargate). Asimismo, se exigió habilitar la correlación cross-signal mediante el identificador `trace_id` del estándar W3C TraceContext y medir de forma empírica la penalización computacional (overhead) inducida por la instrumentación frente a una línea base no instrumentada.

Esta versión del informe incorpora las correcciones realizadas tras una revisión técnica de la primera entrega: varios defectos de instrumentación producían métricas plausibles pero incorrectas, y el método de la línea base no desactivaba realmente el SDK. Las evidencias y los resultados de las secciones de visualización y de overhead se volvieron a tomar; el detalle de las correcciones se presenta al final de la Fase 4.

Arquitectura objetivo y flujo transaccional

La topología de la aplicación responde al patrón de orquestación de servicios:

1. **service-a (orquestador HTTP):** expone el endpoint público `GET /order/{order_id}` sobre FastAPI. Al recibir la solicitud inicia el contexto de

traza distribuida, consulta la tabla `orders` en PostgreSQL mediante Psycopg2 y realiza una llamada HTTP asíncrona hacia service-b con HTTPX.

2. **service-b (servicio de inventario):** expone el endpoint interno `GET /inventory/{product_id}`. Extrae el encabezado `traceparent` propagado por el orquestador, continúa el mismo `trace_id` y consulta la tabla `inventory`.
3. **OpenTelemetry Collector Contrib (v0.103.0):** agente intermedio que recibe señales por OTLP (4317 gRPC y 4318 HTTP), ejecuta el pipeline de procesamiento y exporta hacia Jaeger y Tempo (trazas), Prometheus (métricas) y Loki o Google Cloud Logging (logs).

Fase 1 — Instrumentación con OTel SDK en Python

Identidad del servicio mediante el Resource

El objeto `Resource` define el origen de la telemetría y viaja de forma inmutable en cada traza, métrica y línea de log. En ambos microservicios se configuró con el nombre del servicio, la versión, el entorno, el proveedor de nube y un identificador de instancia por proceso:

```
resource = Resource.create({
    SERVICE_NAME:    "service-a",
    SERVICE_VERSION: APP_VERSION,
    "deployment.environment": ENV,
    "cloud.provider": os.getenv("CLOUD_PROVIDER", "gcp"),
    "host.name":      os.getenv("HOSTNAME", "local"),
    "service.instance.id": f"{os.getenv('HOSTNAME', 'local')}-{os.getpid()}",
})
```

El atributo `service.instance.id` es indispensable porque cada contenedor ejecuta `uvicorn --workers 2`: sin una identidad por proceso, ambos workers exportan series con las mismas etiquetas y sus contadores acumulados se sobrescriben en el Collector, falseando los SLIs.

Toda la inicialización del SDK está condicionada por la variable `OTEL_ENABLED` (por defecto `true`). Con `OTEL_ENABLED=false` no se registra ningún provider ni instrumentor y la API de OpenTelemetry opera en modo *no-op*, de modo que el código de negocio permanece idéntico. Este interruptor es el que permite medir el overhead real en la Fase 4.

TracerProvider y selección de procesadores de spans

Una decisión de diseño crítica es la elección del procesador de spans. `SimpleSpanProcessor` realiza una exportación bloqueante por cada span finalizado; en un endpoint que genera spans para la ruta HTTP, las consultas SQL y la llamada saliente, añadiría decenas de milisegundos por solicitud. Por ello se implementó `BatchSpanProcessor` con un `OTLPSpanExporter` sobre gRPC: retiene los spans en un búfer en memoria y los despacha en lotes de hasta 512 elementos cada 5 segundos, aislando el hilo de atención de solicitudes del tráfico de telemetría.

Auto-instrumentación y spans personalizados

Se combinaron dos niveles de instrumentación:

1. **Auto-instrumentación:** `FastAPIInstrumentor.instrument_app(app)` genera el span raíz (SERVER) de cada solicitud y extrae el encabezado `traceparent`; `HTTPXClientInstrumentor` inyecta el encabezado W3C en las llamadas a service-b; `Psycopg2Instrumentor` registra cada sentencia SQL con su `db.statement`, y `SystemMetricsInstrumentor` emite el CPU y la memoria de cada proceso. Se instrumenta la *instancia* de la aplicación y no la clase: `FastAPIInstrumentor().instrument()` reemplaza la clase dentro del módulo `fastapi`, pero como `main.py` importa `FastAPI` antes de esa llamada, la aplicación se construía con la clase original y quedaba sin instrumentar.
2. **Spans de lógica de negocio:** la auto-instrumentación captura la infraestructura pero omite el contexto de dominio, por lo que se añadieron `fetch.order.db` (atributos `order.id` y `order.status`) y `call.service-b.inventory` en el orquestador, e `inventory.db.fetch` en el servicio de inventario.

MeterProvider y los cuatro indicadores de nivel de servicio (SLIs)

Siguiendo los Golden Signals de Google SRE (Ewaschuk, 2016), se configuró un `MeterProvider` con un `PeriodicExportingMetricReader` que envía las mediciones al Collector cada 15 segundos:

1. **Disponibilidad (SLI-1):** contador `http_requests_total` con la etiqueta `status` (`2xx`, `4xx`, `5xx`). Se incrementa una sola vez por solicitud, en el bloque `finally`, con la clase de estado real de la respuesta.
2. **Latencia (SLI-2):** histograma `http_request_duration_seconds` para estimar p50, p95 y p99. Se declararon cubetas explícitas en segundos (de 5 ms a 10 s) mediante `explicit_bucket_boundaries_advisory`: las cubetas por defecto del SDK (0, 5, 10, 25…) están pensadas para milisegundos, y con valores en segundos toda la distribución caía en la primera cubeta, produciendo un p99 artificial de ~4,95 s.
3. **Tasa de error (SLI-3):** proporción de respuestas 5xx sobre el total.
4. **Saturación (SLI-4):** `UpDownCounter` `http_active_requests`, que rastrea las solicitudes en curso.

Logging estructurado y el trace_id como pivote de correlación

Los logs se emiten por dos vías. La primera es la clase `OtelJsonFormatter`, basada en `pythonjsonlogger`, que consulta el span activo y escribe en la salida estándar un JSON con el `trace_id` (32 dígitos hexadecimales) y el `span_id` (16 dígitos). La segunda es un `LoggerProvider` con `OTLPLogExporter`: el `LoggingHandler` adjunta de forma nativa el contexto del span a cada registro y lo envía al Collector por el mismo canal que las trazas y las métricas.

```
logger_provider = LoggerProvider(resource=resource)
logger_provider.add_log_record_processor(
    BatchLogRecordProcessor(OTLPLogExporter(endpoint=OTEL_ENDPOINT, insecure=True)))
set_logger_provider(logger_provider)
handlers.append(LoggingHandler(level=logging.INFO, logger_provider=logger_provider))
```

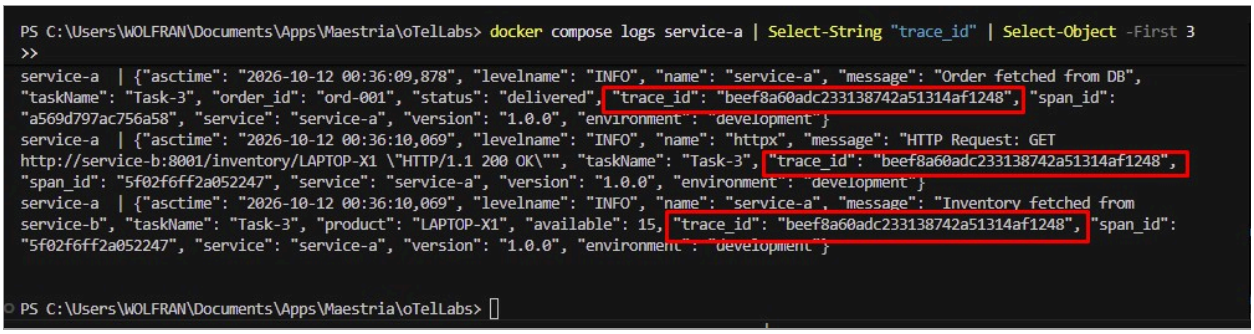


Figura 1. Salida de registros estructurados en formato JSON con inclusión de `trace_id` y `span_id` en service-a.

Fase 2 — Despliegue y configuración del OTel Collector

El OpenTelemetry Collector actúa como concentrador y enrutador agnóstico de telemetría. La configuración `collector-config.yaml` implementa tres pipelines independientes para trazas, métricas y logs.

Orden e importancia de los procesadores

1. **memory_limiter (primer procesador):** límite de 400 MiB, margen de pico de 100 MiB y verificación cada 0,5 s. Ante una tormenta de telemetría descarta datos de forma controlada antes de que el OOM Killer elimine el proceso: un agente caído suspende toda la visibilidad de la plataforma.
2. **resource y resourcedetection:** insertan atributos de entorno (`deployment.environment`, `cloud.region`) y detectan metadatos de GKE o ECS.
3. **filter/health:** excluye las trazas de `/health`; los sondeos de liveness y readiness generan cientos de trazas por minuto sin valor diagnóstico.
4. **batch (último procesador):** agrupa hasta 1024 registros o espera 5 segundos antes de exportar, reduciendo el costo de conexiones de red.

Exporters configurados

- **Trazas:** Jaeger y Tempo vía OTLP gRPC.
- **Métricas:** endpoint de scraping en el puerto 8889 para Prometheus, con `resource_to_telemetry_conversion: true` para convertir `service.name`,

`deployment.environment` y `cloud.provider` en etiquetas. Las métricas internas del Collector (puerto 8888) las recolecta Prometheus directamente.

- **Logs:** Loki 3 mediante `otlphttp/loki`, que almacena `trace_id` y `span_id` como metadatos estructurados, y Google Cloud Logging mediante `googlecloud` en el despliegue GKE.

Para el entorno local se combina un segundo archivo, `collector-config.local.yaml`, que retira `googlecloud` del pipeline de logs: ese exporter exige credenciales de GCP y, sin ellas, impide que el Collector inicie. Al combinar configuraciones, el Collector reemplaza las listas completas, y un exporter definido pero sin pipeline no se inicia. Así, cambiar de backend es una decisión de configuración y no de código.

Fase 3 — Backends, visualización y correlación cross-signal

Propagación de contexto W3C en Jaeger UI

Al consultar el endpoint `/order/{order_id}`, Jaeger muestra que el contexto viaja de extremo a extremo bajo un único identificador. Como se aprecia en la Figura 2, cada traza agrupa 11 spans —siete de service-a y cuatro de service-b—: el span raíz HTTP, la consulta a `orders`, la llamada HTTPX, la recepción en service-b y la consulta a `inventory`. La presencia de spans de ambos servicios en una misma traza demuestra que la inyección y extracción del encabezado `traceparent` funciona.

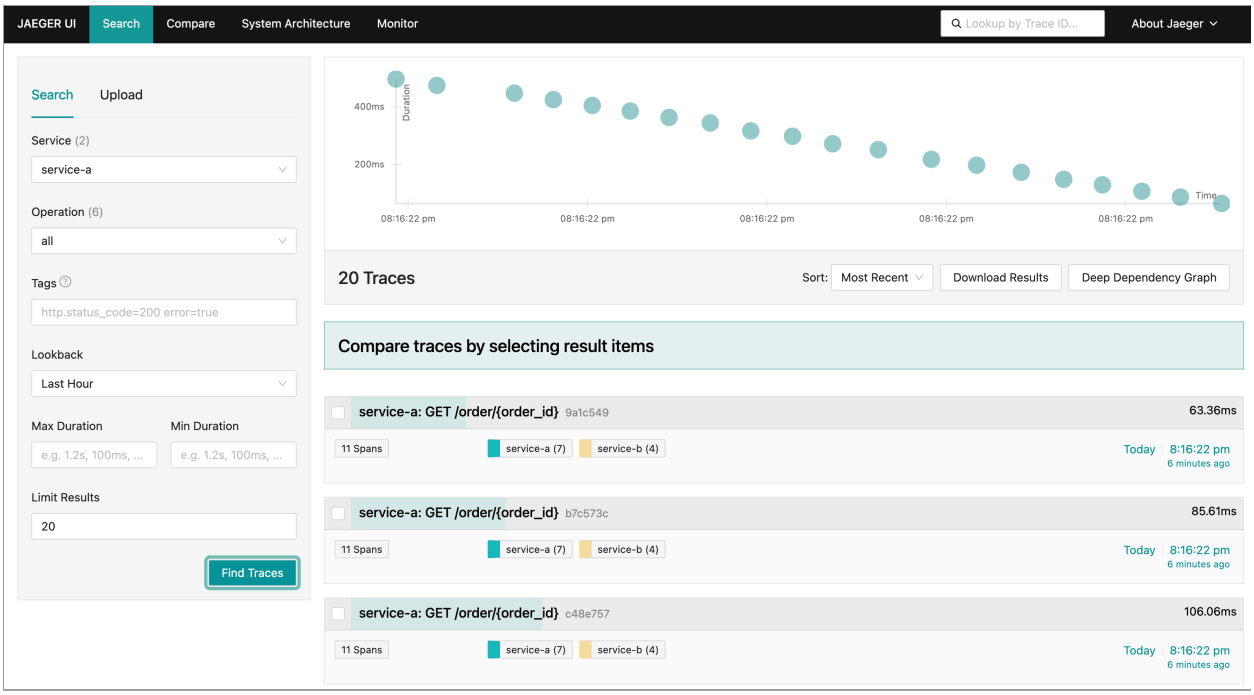


Figura 2. Búsqueda en Jaeger UI: cada traza de `GET /order/{order_id}` agrupa 11 spans de service-a (7) y service-b (4) bajo un único `trace_id`.

Métricas y scraping en Prometheus

Prometheus raspa periódicamente el puerto 8889 del Collector, que expone las métricas exportadas por los SDK con el prefijo `otelcol_`, y el puerto 8888 con las métricas internas del propio Collector.

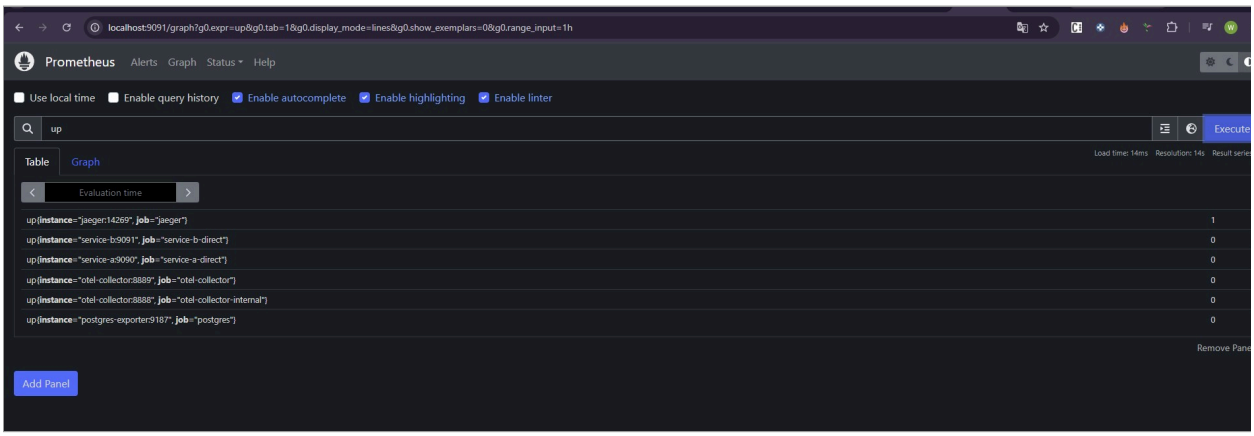


Figura 3. Estado de los targets de scraping y evaluación de métricas OTel en Prometheus UI.

Dashboard de Grafana y paneles SLI

El dashboard `otel-lab-main` contiene los paneles solicitados: disponibilidad (SLI-1, meta ≥ 99,5 %), latencias p50/p95/p99 (SLI-2), tasa de errores 5xx (SLI-3), throughput (SLI-4), CPU de cada servicio frente al Collector y salud del Collector (spans enviados, fallidos, en cola y descartados). Se añadieron además un panel de Burn Rate sobre una ventana de una hora y una tabla de llamadas entre servicios. Las consultas de tasa de error y Burn Rate devuelven 0 cuando no hay respuestas 5xx, para distinguir la ausencia de errores de la ausencia de datos. La Figura 4 se tomó durante una corrida de carga con 75 usuarios concurrentes.

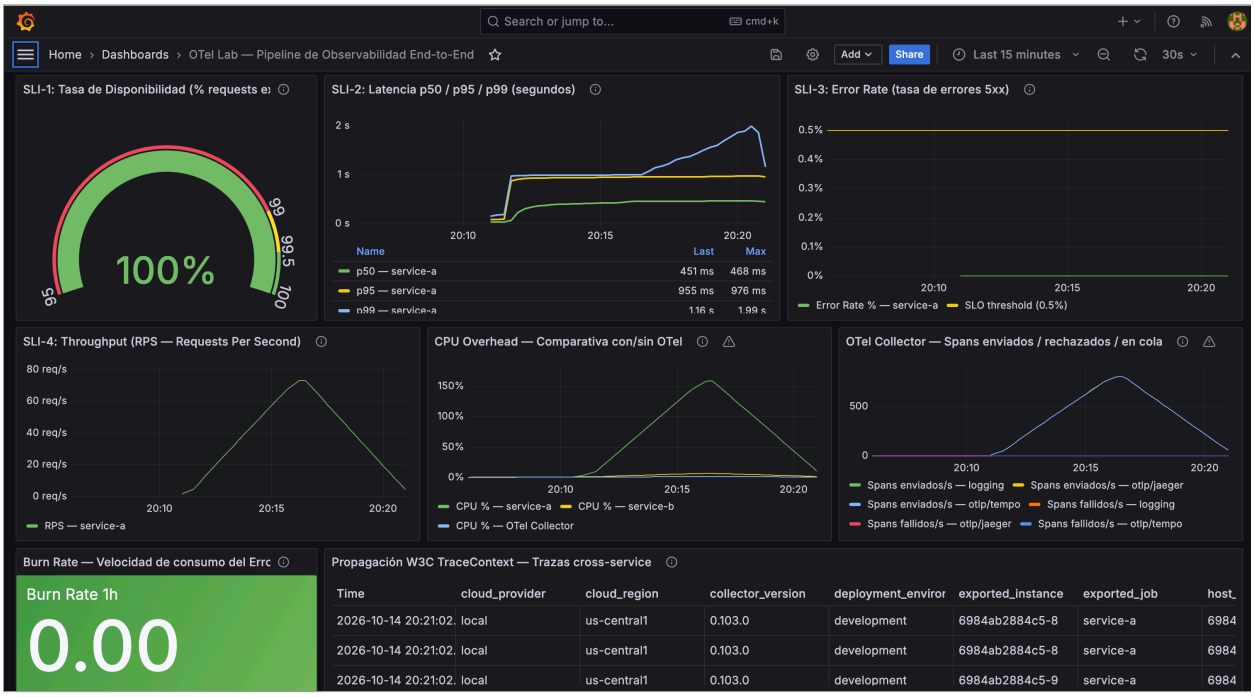


Figura 4. Dashboard en Grafana durante una prueba de carga: cuatro SLIs, CPU por servicio, salud del Collector, Burn Rate y propagación entre servicios.

Correlación cross-signal en Grafana Explore

Los logs OTLP llegan a Loki con el `trace_id` y el `span_id` como metadatos estructurados, junto con los atributos del `Resource`. La consulta `{service_name="service-a"} | trace_id="<id>"` devuelve los logs de una solicitud concreta. El datasource de Loki define un campo derivado sobre `trace_id` que convierte cada valor en un enlace a la traza en Jaeger (log → traza); a la inversa, el datasource de Jaeger configura `tracesToLogsV2` para abrir

desde un span los logs con el mismo identificador (traza → log). El operador puede así pasar de un pico en el panel de latencia a la traza y, de ahí, a la línea de log que describe la causa.

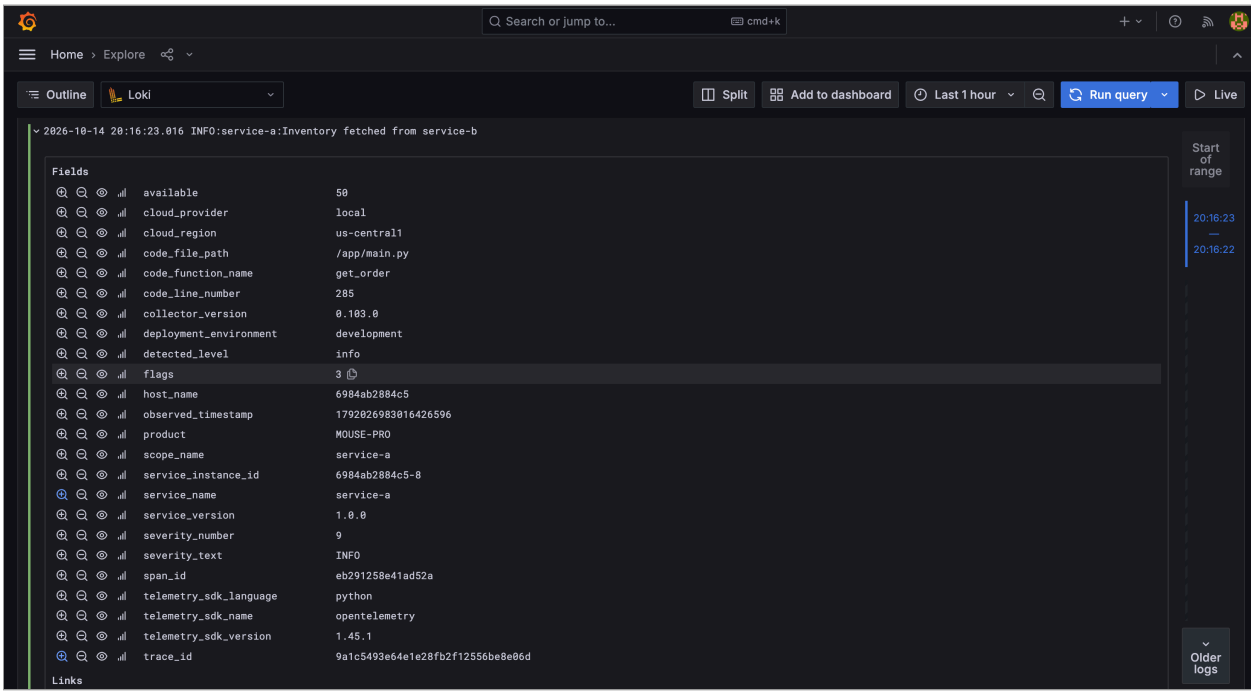


Figura 5. Registro de service-a en Grafana Explore (Loki): el log incluye `trace_id`, `span_id` y los atributos del `Resource`; el campo `trace_id` enlaza con la traza en Jaeger.

Fase 4 — Análisis empírico de overhead con k6

Diseño experimental y perfil de carga

Se ejecutaron pruebas de carga con k6 (v2.2.0) sobre la misma infraestructura y el mismo código, variando únicamente la variable `OTEL_ENABLED` de los servicios:

1. **Línea base (sin OTel, `OTEL_ENABLED=false`):** no se registran providers ni instrumentors. Se conserva el logging JSON a `stdout`, presente en ambos escenarios.
2. **Con OTel SDK (`OTEL_ENABLED=true`):** trazas, métricas y logs exportados por OTLP al Collector.

En la primera entrega, la línea base se obtenía deteniendo el Collector. Ese método no es válido: el SDK sigue creando spans y reintentando su exportación, por lo que no mide el costo de

la instrumentación, y explicaba que la corrida instrumentada pareciera más rápida que la línea base. Esos resultados se descartaron.

El perfil de carga sigue la recomendación del laboratorio: un calentamiento de 1 a 10 usuarios virtuales durante 60 segundos, seguido de **75 usuarios virtuales constantes durante 5 minutos**, con una pausa aleatoria de 100 a 600 ms entre solicitudes. El escenario de pico de 200 usuarios queda como prueba de estrés opcional y no forma parte de la comparación, porque satura la base de datos y su efecto domina el p99. Durante los cinco minutos de carga sostenida, `collect_stats.py` muestrea `docker stats` de service-a, service-b y el Collector; `analyze_overhead.py` consolida esos datos con los resultados de k6. El entorno fue Docker Compose (Docker 27.4, máquina virtual de 8 vCPU y 7,6 GiB) sobre un equipo Apple M1 Pro, con k6 ejecutándose en el mismo equipo.

Tabla 1

Comparación de métricas de rendimiento y recursos entre la línea base y la aplicación con OTel SDK

Métrica	Sin OTel (baseline)	Con OTel SDK	Diferencia absoluta	Overhead (%)
Latencia promedio	564,1 ms	647,4 ms	+83,3 ms	+14,8 %
Latencia percentil 95 (p95)	804,8 ms	899,5 ms	+94,7 ms	+11,8 %
Latencia percentil 99 (p99)	834,4 ms	1.035,8 ms	+201,4 ms	+24,1 %
Tasa de errores	0,00 %	0,00 %	+0,00 %	—
Throughput	69,5 req/s	63,8 req/s	−5,7 req/s	−8,2 %
Uso medio de CPU (service-a + service-b)	159,0 %	168,1 %	+9,0 pp	+5,7 %
CPU por solicitud	22,9 ms	26,3 ms	+3,5 ms	+15,1 %
Memoria (service-a + service-b)	307,9 MiB	318,0 MiB	+10,1 MiB	+3,3 %
Memoria del OTel Collector	59,8 MiB	107,8 MiB	+48,0 MiB	+80,3 %

Nota. Valores de `results_baseline.json` y `results_otel.json` (k6) y de `stats_baseline.json` y `stats_otel.json` (`collect_stats.py`). Se procesaron 25.086 solicitudes en la línea base y 23.031 con OTel, sin errores. *CPU por solicitud* = CPU medio de ambos servicios dividido por el throughput; aísla el costo del SDK de la saturación. pp = puntos porcentuales.

Interpretación de los resultados

El análisis de los datos experimentales arroja los siguientes hallazgos:

1. **Ambos escenarios operan cerca del techo de CPU de service-a.** El servicio ejecuta dos workers de uvicorn (techo teórico de 200 % de CPU) y promedió 155,8 % sin instrumentación y 160,7 % con OTel. En ese régimen, cualquier costo de CPU adicional se traduce en menos solicitudes atendidas y en más tiempo de espera en cola, por lo que el overhead se manifiesta principalmente en el throughput (−8,2 %) y en los percentiles altos de latencia (+201 ms en p99, +24,1 %).
2. **El costo intrínseco del SDK se expresa mejor como CPU por solicitud.** Dividiendo el CPU consumido por el throughput, cada solicitud pasa de 22,9 ms a 26,3 ms de CPU (+3,5 ms, +15 %). Cada llamada a `/order` genera 11 spans, tres a cuatro registros de log OTLP y varias mediciones de métricas entre los dos servicios. En Python, la creación de spans, la serialización de atributos y el `LoggingHandler` se ejecutan en el hilo de la solicitud; solo la exportación es asíncrona gracias a `BatchSpanProcessor` y `BatchLogRecordProcessor`.
3. **El principal cuello de botella es la aplicación, no OTel.** Al perfilar service-a se midió que crear un `httpx.AsyncClient` nuevo en cada solicitud —por la inicialización del contexto SSL— cuesta 14,9 ms de CPU, aproximadamente dos tercios del costo total por solicitud en la línea base; abrir una conexión `psycopg2` por solicitud suma otros 2,4 ms. Reutilizar un único cliente HTTP y un pool de conexiones elevaría la capacidad del servicio y reduciría el peso relativo de la instrumentación.
4. **Memoria.** El SDK agrega 10 MiB entre ambos servicios (+3,3 %), atribuibles a las colas de spans y logs pendientes de exportación y al estado de los agregadores de métricas. El Collector pasó de 60 a 108 MiB al procesar telemetría real, muy por debajo del límite de 400 MiB del `memory_limiter`.
5. **SLO de latencia.** El umbral de k6 `p(95) < 500 ms` se incumplió en ambos escenarios, lo que confirma que la limitación de capacidad existe con o sin instrumentación.

6. **Veredicto.** Con 75 usuarios concurrentes sobre un servicio limitado por CPU, el overhead de OTel no es despreciable: la instrumentación consume alrededor del 13 % del CPU de cada solicitud. Para producción se recomienda (a) corregir el cuello de botella de la aplicación antes de dimensionar; (b) dimensionar la capacidad con la instrumentación activa, porque la línea base sobreestima la capacidad real; (c) aplicar muestreo en el SDK (`ParentBased(TraceIdRatioBased)`) o *tail sampling* en el Collector cuando el costo por solicitud sea crítico, y (d) revisar el volumen de logs OTLP por solicitud.

Validez de la medición. Una primera corrida con OTel se descartó porque el equipo anfitrión se suspendió durante aproximadamente 80 segundos en mitad de la carga sostenida, lo que distorsionaba el throughput y los percentiles (su registro se conserva como `benchmark/output/k6_otel_descartada_suspension.log`). La corrida reportada se ejecutó con la suspensión del sistema inhibida (`caffeinate`). Una corrida adicional independiente con OTel registró 64,2 req/s y un p99 de 1.039,5 ms, prácticamente idénticos a los reportados, lo que respalda la reproducibilidad de la medición. Dado que k6 se ejecuta en el mismo equipo que los contenedores, los valores absolutos no son extrapolables a un clúster dedicado; una medición concluyente requeriría repetir cada escenario al menos tres veces y reportar la mediana.

Figura 6. Extracto de la salida de consola de k6 para ambos escenarios y tabla generada por `analyze_overhead.py`. Salidas completas en `benchmark/output/`.

```
# k6 run --env INSTRUMENTED=false  (OTEL_ENABLED=false)
   ✗ http_req_duration..............: avg=564.13ms min=1.49ms   med=585.58ms max=1.39s
p(90)=784.77ms p(95)=804.83ms p(99)=834.4ms  p(99.9)=1.22s
   ✓ http_req_failed................: 0.00%  ✓ 0         ✗ 25086
     http_reqs......................: 25086  69.518256/s
     iterations.....................: 25085  69.515485/s
sustained_load ✓ [ 100% ] 75 VUs     5m0s

# k6 run --env INSTRUMENTED=true   (OTEL_ENABLED=true)
   ✗ http_req_duration..............: avg=647.41ms min=5.4ms    med=670.88ms max=1.47s
p(90)=866.26ms p(95)=899.53ms p(99)=1.03s   p(99.9)=1.4s
   ✓ http_req_failed................: 0.00%  ✓ 0         ✗ 23031
     http_reqs......................: 23031  63.831807/s
     iterations.....................: 23030  63.829036/s
sustained_load ✓ [ 100% ] 75 VUs     5m0s

# python3 benchmark/analyze_overhead.py
Tabla para el reporte del laboratorio:
   ┌──────────────────────────┬──────────────┬──────────────┬──────────┐
   │ Métrica                  │ Sin OTel     │ Con OTel     │ Overhead │
   ├──────────────────────────┼──────────────┼──────────────┼──────────┤
   │ Latencia promedio (ms)   │     564.1 ms │     647.4 ms │   +14.8% │
   │ Latencia p95 (ms)        │     804.8 ms │     899.5 ms │   +11.8% │
   │ Latencia p99 (ms)        │     834.4 ms │    1035.8 ms │   +24.1% │
   │ Error rate (%)           │       0.00 % │       0.00 % │        — │
   │ Throughput (req/s)       │     69.5 rps │     63.8 rps │    -8.2% │
   │ CPU apps (%)             │      159.0 % │      168.1 % │    +5.7% │
   │ Memoria apps (MiB)       │    307.9 MiB │    318.0 MiB │    +3.3% │
   └──────────────────────────┴──────────────┴──────────────┴──────────┘
```

Correcciones posteriores a la primera entrega

La revisión técnica de la primera versión identificó los siguientes defectos, todos corregidos antes de volver a tomar las evidencias:

Tabla 2

Defectos de instrumentación detectados, impacto y corrección aplicada

Defecto detectado	Impacto	Corrección
`FastAPIInstrumentor().instrument()` invocado después de importar `FastAPI`	Trazas fragmentadas: sin span raíz y service-b en trazas separadas	`FastAPIInstrumentor.instrument_app(app)` sobre la instancia
Línea base obtenida deteniendo solo el Collector	El SDK seguía activo; el overhead reportado no era real	Variable `OTEL_ENABLED` en ambos servicios
CPU y memoria sin fuente de datos	Cifras no verificables	`collect_stats.py` → `stats_*.json`

Defecto detectado	Impacto	Corrección
`http_requests_total` incrementado 2–3 veces por solicitud	SLI-1 de disponibilidad distorsionado	Un único incremento por solicitud en `finally`
`HTTPStatusError.status_code` (atributo inexistente)	Un fallo de service-b terminaba en un 500 no controlado	`e.response.status_code`
Histogramas en segundos con cubetas por defecto (pensadas para ms)	p99 fijo en ~4,95 s, sin relación con la latencia real	Cubetas explícitas en segundos
Logs solo en `stdout`, sin backend de logs en Grafana	Correlación log ↔ traza imposible en Grafana Explore	`LoggerProvider` OTLP + Loki + campo derivado `trace_id`
Sin etiqueta `service_name` en Prometheus	Variable `$service` del dashboard vacía	`resource_to_telemetry_conversion: true`
Dos workers de uvicorn con la misma identidad	Contadores sobrescritos entre procesos	`service.instance.id` por proceso
Panel de CPU sobre `process_cpu_seconds_total`	Graficaba el CPU de Jaeger, no el de los servicios	`process.cpu.time` vía `SystemMetricsInstrumentor`
Healthcheck con `curl`, ausente en la imagen `python:slim`	Servicios reportados como *unhealthy*	Healthcheck con `urllib` de Python

Tres de estos defectos —las cubetas del histograma, el contador duplicado y la instrumentación inactiva de FastAPI— producían valores verosímiles pero incorrectos. La lección es que la propia telemetría debe validarse contra una fuente independiente antes de definir SLOs sobre ella.

Despliegue multi-cloud e Infraestructura como Código (IaC)

Estrategia en Google Cloud Platform (GKE)

Los manifiestos de Terraform (`terraform/gcp/`) aprovisionan una red VPC, un clúster GKE con Workload Identity y un pool de nodos. El OTel Collector se diseñó con el patrón DaemonSet (un pod por nodo): un único proceso atiende la telemetría de todos los pods del nodo, reduciendo el consumo de memoria frente al modelo sidecar. La autenticación con Cloud Trace y Cloud Logging se resuelve vinculando el ServiceAccount de Kubernetes con una cuenta de servicio de GCP con los roles `roles/cloudtrace.agent` y `roles/logging.logWriter`.

Las Figuras 7 y 8 muestran el clúster `otel-lab` con tres nodos operativos y los pods de service-a, service-b y Jaeger en ejecución. Sin embargo, en esas capturas el DaemonSet del Collector no tenía pods listos (0/3) y no aparece el pod de PostgreSQL, por lo que la telemetría de extremo a extremo se validó en el entorno local (Fases 1 a 4) y no en GKE. Además, el

despliegue en GKE es anterior a las correcciones de instrumentación y debe repetirse con las imágenes actualizadas.

```
C:\Users\WOLFRAN\Documents\Apps\Maestria\oTelLabs>kubectl get pods -n otel-lab
NAME                       READY   STATUS    RESTARTS   AGE
jaeger-84c9b4f8d7-2txs8    1/1     Running   0          15m
service-a-7f4bc945dc-cpgbv   1/1     Running   0          3m23s
service-a-7f4bc945dc-xs8dn   1/1     Running   0          3m25s
service-b-5f878cf8b9-bfqfk   1/1     Running   0          3m24s
service-b-5f878cf8b9-qc6b5   1/1     Running   0          3m21s

C:\Users\WOLFRAN\Documents\Apps\Maestria\oTelLabs>kubectl get services -n otel-lab
NAME                TYPE           CLUSTER-IP       EXTERNAL-IP      PORT(S)                         AGE
jaeger-svc          LoadBalancer   34.118.225.118   136.80.234.127   16686:31863/TCP,14250:31492/TCP   15m
otel-collector-svc  ClusterIP      34.118.228.102   <none>           4317/TCP,4318/TCP,8889/TCP      15m
service-a-svc       LoadBalancer   34.118.234.59    136.114.222.84   8000:32486/TCP,9090:32642/TCP   15m
service-b-svc       ClusterIP      34.118.230.60    <none>           8001/TCP,9091/TCP               15m
```

Figura 7. Pods y servicios en el namespace `otel-lab` de GKE: service-a, service-b y Jaeger en ejecución; no figuran pods del Collector ni de PostgreSQL.

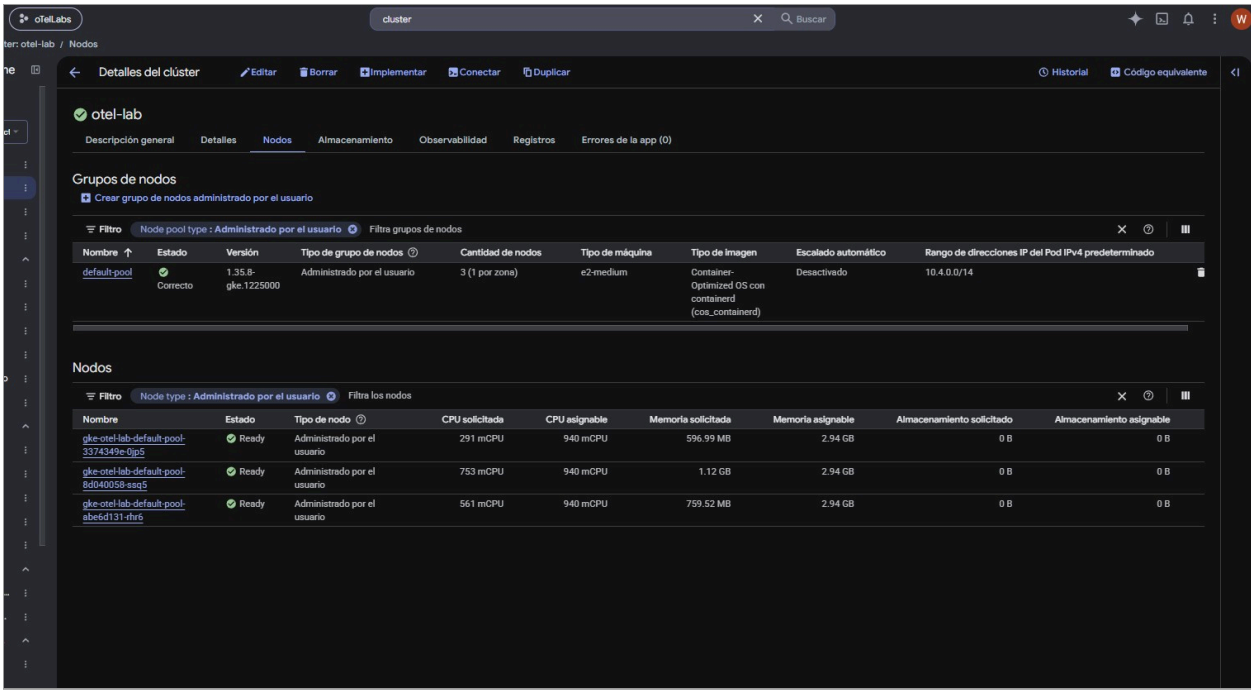


Figura 8. Clúster GKE `otel-lab` en la consola de Google Cloud Platform con su grupo de nodos de tres instancias.

Estrategia en Amazon Web Services (ECS Fargate)

En AWS, Terraform (`terraform/aws/`) crea una VPC multi-AZ, un clúster ECS, un Log Group de CloudWatch y los roles IAM con las políticas `AWSXRayDaemonWriteAccess` y `CloudWatchLogsFullAccess`. Al ser Fargate un modelo serverless sin acceso a los nodos, el

patrón DaemonSet no es aplicable: la *task definition* ejecuta el Collector como sidecar, comunicado con los servicios por `localhost:4317`.

Limitación declarada. El despliegue en AWS no se ejecutó, por lo que no se presentan evidencias de X-Ray ni de CloudWatch. Para completarlo falta agregar `aws_ecs_task_definition` y `aws_ecs_service` al módulo de Terraform, y una configuración del Collector específica para AWS con los exporters `awsxray` y `awscloudwatchlogs`, ya que la configuración actual exporta a Jaeger y Cloud Logging.

Tabla 3

Comparativa arquitectónica de despliegue entre GCP GKE y AWS ECS Fargate

Criterio	Google Cloud Platform (GKE)	Amazon Web Services (ECS Fargate)
Patrón del Collector	DaemonSet (1 pod por nodo)	Sidecar (1 contenedor por tarea)
Mecanismo de autorización	Workload Identity (K8s SA → GCP SA)	IAM Roles for Tasks (Task Role)
Automatización IaC	Terraform + Helm Charts	Terraform + Task Definition JSON
Backend nativo de trazas	Google Cloud Trace	AWS X-Ray
Backend nativo de logs	Google Cloud Logging	Amazon CloudWatch Logs
Impacto en memoria y costos	Menor huella (agente compartido)	Aislamiento por tarea (mayor costo)
Estado en este laboratorio	Clúster y servicios desplegados; Collector sin pods listos	Definido en IaC; sin desplegar

Nota. Comparación de trade-offs operacionales entre el despliegue en nodos Kubernetes compartidos y el despliegue serverless en contenedores Fargate.

Conclusiones

1. **Interoperabilidad sin bloqueo con proveedores.** OpenTelemetry desacopla la lógica de los microservicios de las plataformas de visualización: el mismo código alimenta Jaeger, Tempo, Prometheus, Loki y Cloud Logging, y el cambio de backend se resuelve en la configuración del Collector, como demuestra el archivo de override local que retira Cloud Logging sin tocar el código.
2. **Correlación cross-signal.** La propagación W3C `traceparent` y el registro del `trace_id` en logs OTLP permiten pasar del dashboard a la traza y de la traza al log exacto en Grafana Explore, eliminando la correlación manual de marcas temporales.
3. **El overhead depende del margen de capacidad.** La instrumentación añadió 3,5 ms de CPU por solicitud (+15 %). En un servicio cerca de su techo de CPU, ese costo se traduce en −8,2 % de throughput y +24,1 % de p99. La conclusión de la primera entrega —un overhead de +0,54 %— provenía de una línea base que no desactivaba el SDK. La capacidad debe dimensionarse con la instrumentación activa.
4. **La telemetría también debe validarse.** Defectos como cubetas de histograma inadecuadas, contadores duplicados o una instrumentación que no se activa producen métricas verosímiles pero falsas, más peligrosas que la ausencia de métricas porque generan confianza injustificada.
5. **Resiliencia operativa del Collector.** Situar `memory_limiter` como primer procesador y filtrar los health checks antes del empaquetado en lotes protege la disponibilidad del agente de telemetría ante picos de carga.

Referencias

Amazon Web Services. (2024). *AWS Distro for OpenTelemetry on Amazon ECS*. https://docs.aws.amazon.com/ecs/latest/developerguide/opentelemetry.html

Beyer, B., Jones, C., Petoff, J. y Murphy, N. R. (Eds.). (2018). *The site reliability workbook: Practical ways to implement SRE*. O'Reilly Media.

Cloud Native Computing Foundation. (2024). *OpenTelemetry specification*. https://opentelemetry.io/docs/specs/otel/

Ewaschuk, R. (2016). Monitoring distributed systems. En B. Beyer, C. Jones, J. Petoff y N. R. Murphy (Eds.), *Site reliability engineering: How Google runs production systems* (pp. 57–67). O'Reilly Media.

Google Cloud. (2024). *Observability architecture: Designing telemetry pipelines with OpenTelemetry Collector*. https://cloud.google.com/architecture/observability-opentelemetry

Grafana Labs. (2024). *Trace integration in Explore*. https://grafana.com/docs/grafana/latest/explore/trace-integration/

Grafana Labs. (2024). *Ingesting logs to Loki using OpenTelemetry Collector*. https://grafana.com/docs/loki/latest/send-data/otel/

Grafana Labs. (2024). *k6 documentation*. https://grafana.com/docs/k6/latest/

Ochoa Paipilla, M. F. (2026, 8 de octubre). *Chaos engineering, métricas SRE y baseline dinámico de alertas* [Sesión de clase]. Maestría en Arquitectura de Software, Universidad de La Sabana.

OpenTelemetry Authors. (2024). *OpenTelemetry Python documentation*. https://opentelemetry-python.readthedocs.io/

Sachto, A. y Walcer, A. (2022). *Anatomy of an incident*. Google Site Reliability Engineering. https://sre.google/resources/practices-and-processes/anatomy-of-an-incident/

World Wide Web Consortium. (2021). *Trace context* (W3C Recommendation). https://www.w3.org/TR/trace-context/